PRJ301: Java Web Application Development

Slot 13, 14: Expression Language & JSTL

VinhVT9@fe.edu.vn

Spring 2026

FPT University Can Tho

Learning Objectives

By the end of this slot, students will be able to:

- Explain **what Expression Language (EL) is** and why it replaces scriptlets.
- Use **EL syntax** `${expr}` to access beans, collections, and parameters.
- Apply **EL operators**: arithmetic, relational, logical, and `empty`.
- Use all **EL implicit objects** (`param`, `requestScope`, `sessionScope`, ...).
- Explain **what JSTL is** and how to include it in a project.
- Use **JSTL Core** tags: `c:out`, `c:set`, `c:if`, `c:choose`, `c:forEach`.
- Apply **JSTL fmt** tags for date and number formatting.
- Use **JSTL fn** functions for string manipulation.
- Combine **EL + JSTL** to build clean, scriptlet-free JSP views.

FPT Education
FPT UNIVERSITY

Outline

Part 1: Expression Language (EL)

1. What is EL and Why?
2. EL Syntax and Operators
3. EL Implicit Objects
4. Accessing Collections and Beans
5. EL Functions

Part 2: JSTL

6. What is JSTL?
7. JSTL Core Tags
8. JSTL fmt Tags
9. JSTL fn Functions
10. EL + JSTL Combined
11. Classwork

FPT Education
FPT UNIVERSITY

Part 1: Expression Language (EL)

1. What is EL and Why?

The Problem with Scriptlets

Consider displaying a bean property using classic JSP:

- You have to cast, check null, and call a getter — all in the JSP.
- This mixes Java logic with HTML presentation.

Old Way (Scriptlet)

```
<%@ page import="com.example.User" %>
<%
  User u = (User) request.getAttribute("user");
  out.print(u.getFirstName());
%>
```

Better Way (EL)

```
${user.firstName}
```

EL is **concise**, **null-safe**, and **designer-friendly**.

FPT Education
FPT UNIVERSITY

What is Expression Language (EL)?

- **EL** is a scripting language built into JSP (since JSP 2.0 / Jakarta EE).
- It provides **read access** to objects stored in various scopes.
- EL was originally from **JSTL** (called SPEL), then unified into the JSP spec (JSP 2.1 + JSF 1.2 $\rightarrow$ Unified EL).
- EL expressions are wrapped in `${ ...}`.

Feature	Description
Bean property access	`${user.firstName}`
Collection element	`${list[0]}`, `${map["key"]}`
Request parameters	`${param.username}`
Operators	`${a + b}`, `${x > 5}`
Null safety	Missing values return empty string

FPT Education
FPT UNIVERSITY

2. EL Syntax and Operators

EL Syntax: ${...}

- All EL expressions are enclosed in ${ ...}.
- Two notation styles for property/element access:

Dot notation (bean property):

```
${user.firstName}
${company.president.name}
```

Bracket notation (any key):

```
${user["firstName"]}
${list[0]}
${map["key-name"]}
```

- Dot notation requires **valid Java identifiers** as property names.
- Bracket notation works for **arrays, lists, maps**, and keys with special characters.

Null Safety
If `user` is `null`, `${user.firstName}` returns **empty string** — no exception thrown.

FPT Education
FPT UNIVERSITY

EL Operators

Category	Operators	Example
Arithmetic	`+ - * / div % mod`	`${a + b}, ${10 div 3}`
Relational	`== eq != ne < lt > gt`	`${age >= 18}`
Relational	`<= le >= ge`	`${score lt 50}`
Logical	`&& and || or ! not`	`${a && b}`
Empty	`empty`	`${empty list}`
Conditional	`? :`	`${x>0 ? "pos" : "neg"}`

The `empty` operator returns `true` for:

- `null` values
- Empty `String` (`""`)
- Empty array, empty `List`, empty `Map`

```
${empty sessionScope.cart}
${not empty param.username}
${score >= 50 ? "Pass" : "Fail"}
```

FPT Education
FPT UNIVERSITY

3. EL Implicit Objects

EL Implicit Objects

EL provides **11 implicit objects** (different from JSP implicit objects!):

Object	Description
`pageScope`	Map of page-scoped attributes
`requestScope`	Map of request-scoped attributes
`sessionScope`	Map of session-scoped attributes
`applicationScope`	Map of application-scoped attributes
`param`	Map of single request parameter values
`paramValues`	Map of request parameter value arrays
`header`	Map of single HTTP header values
`headerValues`	Map of HTTP header value arrays
`cookie`	Map of cookie objects by name
`initParam`	Map of context init parameters
`pageContext`	The `PageContext` object (access to all JSP objects)

FPT Education
FPT UNIVERSITY

EL Implicit Objects: Examples

```
<%-- Request parameter --%>
Hello, ${param.username}!

<%-- Request/Session attributes --%>
Cart: ${requestScope.itemCount}
User: ${sessionScope.currentUser}

<%-- Cookie value --%>
Visit: ${cookie["lastVisit"].value}

<%-- Context init param (web.xml) --%>
Version: ${initParam.appVersion}

<%-- Request info via pageContext --%>
Method: ${pageContext.request.method}
```

FPT Education
FPT UNIVERSITY

EL Scope Search Order

When you write `${someBean}` without specifying a scope:



- EL searches from **narrowest** (page) to **widest** (application) scope.
- First match wins.
- To be explicit, specify scope: `${sessionScope.user}`.

```
${someBean}     <%-- searches all scopes --%>
${sessionScope.user} <%-- session only --%>
```

FPT Education
FPT UNIVERSITY

4. Accessing Collections and Beans

Accessing JavaBeans with EL

Given a `Customer` bean with fields `firstName`, `lastName`, `address`:

```
Customer c = dao.findById(id);
request.setAttribute("customer", c);
dispatcher.forward(request, response);
```

Listing 1: Servlet sets the bean

```
<p>Name: ${customer.firstName}
         ${customer.lastName}</p>
<p>City: ${customer.address.city}</p>
```

Listing 2: JSP reads the bean with EL

- EL calls the **getter** method automatically (`getFirstName()`).
- Chaining works: `${customer.address.city}` calls `customer.getAddress().getCity()`.

FPT Education
FPT UNIVERSITY

Accessing Collections with EL

```
// Servlet code
String[] colors = {"red","green","blue"};
request.setAttribute("colors", colors);

List<String> fruits = List.of("apple","banana");
request.setAttribute("fruits", fruits);

Map<String,String> cfg = new HashMap<>();
cfg.put("theme", "dark");
request.setAttribute("cfg", cfg);
```

```
<%-- Array / List by index --%>
${colors[0]}    <%-- red --%>
${fruits[1]}    <%-- banana --%>

<%-- Map by key --%>
${cfg["theme"]}    <%-- dark --%>
${cfg.theme}       <%-- dark (dot notation) --%>
```

FPT Education
FPT UNIVERSITY

5. EL Functions

EL Functions

EL supports **static method calls** declared via tag libraries.

- Functions are invoked as `${prefix:functionName(args)}`.
- Most commonly used with the **JSTL** `fn:` **library** (covered next).

```
<%@ taglib uri=
  "http://java.sun.com/jsp/jstl/functions"
  prefix="fn" %>

Length: ${fn:length(itemList)}
Upper:  ${fn:toUpperCase(param.name)}
Sub:    ${fn:substring(s, 0, 5)}
```

Key Point
EL functions require a `taglib` declaration. The `fn:` functions come from JSTL — we will cover them in detail in the JSTL section.

FPT Education
FPT UNIVERSITY

Part 2: JSTL

6. What is JSTL?

What is JSTL?

- **JSTL** (Jakarta Standard Tag Library, formerly JavaServer Pages Standard Tag Library) is a collection of **custom JSP tags**.
- Goal: replace Java scriptlets in JSP with **HTML-like tags**.
- Part of the Jakarta EE specification.
- Provides five tag libraries:

Library	URI prefix	Description
Core	`c:`	Variables, conditions, loops, URLs
Formatting	`fmt:`	Dates, numbers, i18n
SQL	`sql:`	Database queries (not recommended in MVC)
XML	`x:`	XML processing
Functions	`fn:`	String functions

FPT Education
FPT UNIVERSITY

JSTL Setup

Step 1: Add JSTL JAR to your Maven project:

```
<!-- pom.xml -->
<dependency>
  <groupId>jakarta.servlet.jsp.jstl</groupId>
  <artifactId>
    jakarta.servlet.jsp.jstl-api
  </artifactId>
  <version>3.0.0</version>
</dependency>
<dependency>
  <groupId>org.glassfish.web</groupId>
  <artifactId>
    jakarta.servlet.jsp.jstl
  </artifactId>
  <version>3.0.1</version>
</dependency>
```

Step 2: Declare the taglib at the top of each JSP:

```
<%@ taglib
  uri="http://java.sun.com/jsp/jstl/core"
  prefix="c" %>
```

FPT Education
FPT UNIVERSITY

7. JSTL Core Tags

Core: c:out and c:set

c:out — Output a value with XSS escaping:

```
<%-- Safe output (escapes HTML chars) --%>
<c:out value="${user.name}" />
<c:out value="${param.msg}"
       default="No message" />
```

c:set — Set a scoped variable:

```
<%-- Set page-scoped variable --%>
<c:set var="greeting"
       value="Hello World!" />

<%-- Set bean property --%>
<c:set target="${user}"
       property="role"
       value="admin" />
```

- scope attribute: page (default), request, session, application

FPT Education
FPT UNIVERSITY

Core: c:remove and c:if

c:remove — Remove a scoped variable:

```
<c:remove var="tempData" scope="session"/>
```

c:if — Simple conditional (no else branch):

```
<c:if test="${sessionScope.user != null}">
  <p>Welcome, ${sessionScope.user.name}!</p>
</c:if>

<c:if test="${param.age ge 18}">
  <p>Access granted.</p>
</c:if>

<%-- Capture result for reuse --%>
<c:if test="${empty cart}"
      var="emptyCart" scope="request">
  <p>Your cart is empty.</p>
</c:if>
```

FPT Education
FPT UNIVERSITY

Core: c:choose / c:when / c:otherwise

Use c:choose for **multi-branch** conditionals (like if-else if-else):

```
<c:choose>
  <c:when test="${score >= 90}">
    <span class="grade-a">A - Excellent!</span>
  </c:when>
  <c:when test="${score >= 75}">
    <span class="grade-b">B - Good</span>
  </c:when>
  <c:when test="${score >= 50}">
    <span class="grade-c">C - Pass</span>
  </c:when>
  <c:otherwise>
    <span class="grade-f">F - Fail</span>
  </c:otherwise>
</c:choose>
```

- Only the **first matching** c:when executes.
- c:otherwise is the default (like else).

FPT Education
FPT UNIVERSITY

Core: c:forEach — Iterating Collections

```jsp
<table>
  <tr><th>Name</th><th>Price</th></tr>
  <c:forEach var="p" items="${productList}">
    <tr>
      <td>${p.name}</td>
      <td>${p.price}</td>
    </tr>
  </c:forEach>
</table>
```

Listing 3: Iterating a list of products

With varStatus (loop counter/status):

```jsp
<c:forEach var="item" items="${items}"
           varStatus="st"
           begin="0" end="4" step="1">
  <p>${st.index}: ${item.name}
     ${st.first ? "(first)" : ""}
     ${st.last  ? "(last)"  : ""}
  </p>
</c:forEach>
```

FPT Education
FPT UNIVERSITY

Core: c:forTokens, c:url, c:redirect

c:forTokens — Split and iterate a delimited string:

```
<c:forTokens var="color"
             items="red,green,blue"
             delims=",">
  <span>${color}</span>
</c:forTokens>
```

c:url — Build a URL with session encoding:

```
<a href="<c:url value='/product'>
  <c:param name='id' value='42'/>
</c:url>">View Product</a>
```

c:redirect — Send a redirect response:

```
<c:if test="${empty sessionScope.user}">
  <c:redirect url="/login.jsp"/>
</c:if>
```

FPT Education
FPT UNIVERSITY

8. JSTL fmt Tags

JSTL fmt: Date and Number Formatting

Declare the fmt library first:

```
<%@ taglib uri=
  "http://java.sun.com/jsp/jstl/fmt"
  prefix="fmt" %>
```

Date formatting:

```
<%-- Full date --%>
<fmt:formatDate value="${order.date}"
  pattern="dd/MM/yyyy HH:mm" />

<%-- Short style --%>
<fmt:formatDate value="${dob}"
  type="date" dateStyle="medium" />
```

Number formatting:

```
Price: <fmt:formatNumber value="${price}"
         type="currency"
         currencySymbol="$" />

Rate: <fmt:formatNumber value="${rate}"
        type="percent"
        maxFractionDigits="1" />
```

FPT Education
FPT UNIVERSITY

JSTL fmt: Locale and i18n

`fmt:setLocale` — Set the locale for formatting:

```
<fmt:setLocale value="vi_VN"/>
<fmt:formatNumber value="1234567.89"
  type="currency"/>
<%-- Outputs: 1.234.567,89 VND --%>
```

`fmt:bundle` / `fmt:message` — Internationalized messages:

```
<fmt:bundle basename="messages">
  <fmt:message key="welcome.title"/>
  <fmt:message key="login.button"/>
</fmt:bundle>
```

Use Case
Store UI strings in `messages_en.properties`, `messages_vi.properties`, etc. The correct file is loaded automatically based on locale.

FPT Education
FPT UNIVERSITY

9. JSTL fn Functions

JSTL fn: String Functions

Declare the functions library:

```
<%@ taglib uri=
  "http://java.sun.com/jsp/jstl/functions"
  prefix="fn" %>
```

Function	Description & Example
`fn:length(obj)`	Length of string/collection: `${fn:length(list)}`
`fn:toUpperCase(s)`	`${fn:toUpperCase("hello")}` → HELLO
`fn:toLowerCase(s)`	`${fn:toLowerCase("HELLO")}` → hello
`fn:substring(s,b,e)`	`${fn:substring("abcde",1,3)}` → bc
`fn:contains(s,sub)`	`${fn:contains(name,"admin")}`
`fn:startsWith(s,p)`	`${fn:startsWith(url,"/api")}`
`fn:replace(s,b,a)`	Replace all occurrences
`fn:split(s,delim)`	Split to array
`fn:join(arr,sep)`	Join array to string
`fn:trim(s)`	Remove leading/trailing whitespace

FPT Education
FPT UNIVERSITY

JSTL fn: Practical Examples

```jsp
<%@ taglib uri=
  "http://java.sun.com/jsp/jstl/functions"
  prefix="fn" %>

<%-- Show item count --%>
<p>Total items: ${fn:length(productList)}</p>

<%-- Format name --%>
<p>Hello,
  ${fn:toUpperCase(param.firstName)}!</p>

<%-- Truncate long description --%>
<c:set var="desc" value="${product.desc}"/>
<c:if test="${fn:length(desc) > 50}">
  ${fn:substring(desc,0,50)}...
</c:if>

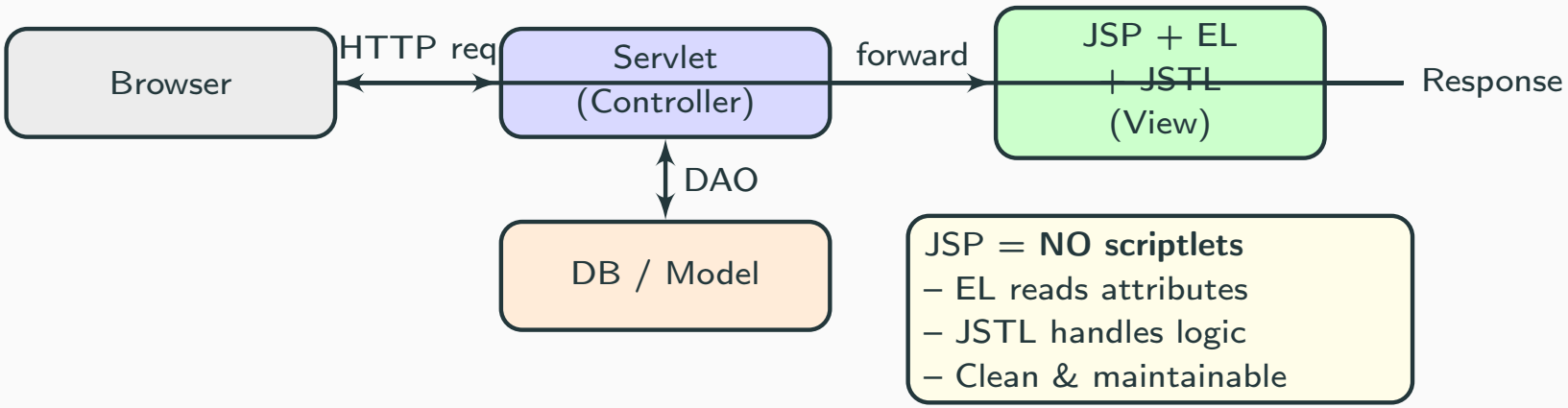
<%-- Check role --%>
<c:if test="${fn:contains(user.roles,'ADMIN')}">
  <a href="/admin">Admin Panel</a>
</c:if>
```

Listing 4: fn functions in practice

FPT Education
FPT UNIVERSITY

10. EL + JSTL Combined

EL + JSTL: The Clean JSP Pattern



The Golden Rule

Servlet processes data and sets request/session attributes.

JSP + EL + JSTL only *reads* and *displays* data.

FPT Education
FPT UNIVERSITY

Complete Example: Product List Page

```
<%@ taglib uri=
  "http://java.sun.com/jsp/jstl/core"
  prefix="c" %>
<%@ taglib uri=
  "http://java.sun.com/jsp/jstl/fmt"
  prefix="fmt" %>
<!DOCTYPE html><html>
<head><title>Products</title></head>
<body>
<h2>${requestScope.products.size()} items</h2>
<c:choose>
<c:when test="${empty products}">
  <p>No products found.</p>
</c:when>
<c:otherwise>
<table border="1">
<tr><th>#</th><th>Name</th><th>Price</th></tr>
<c:forEach var="p" items="${products}"
  varStatus="s">
<tr>
  <td>${s.count}</td>
  <td><c:out value="${p.name}"/></td>
  <td><fmt:formatNumber value="${p.price}"
    type="currency" currencySymbol="$"/></td>
</tr>
</c:forEach>
</table>
</c:otherwise>
</c:choose>
</body></html>
```

Listing 5: productList.jsp

FPT Education
FPT UNIVERSITY

11. Classwork

Classwork 5: EL + JSTL Practice

Setup
Create a new Maven web project: `EL_JSTL_Demo`
Add JSTL dependency to `pom.xml`.

Task A: EL Basics

1. Create a `StudentServlet` that puts a `Student` bean (name, gpa, major) into `requestScope`.
2. In `student.jsp`: display all fields using EL.
3. Display GPA with `fmt:formatNumber` (2 decimal places).
4. Show "Dean's List" only if `gpa >= 3.5` using `c:if`.

Task B: JSTL Loop + Condition

1. Create a `CourseServlet` that builds a `List<Course>` and forwards to `courses.jsp`.
2. Use `c:forEach` to render a table with row counter.
3. Use `c:choose` to color rows: credits ≥ 3 = yellow, else white.

FPT Education
FPT UNIVERSITY

Classwork 5: Task C – Full MVC with EL+JSTL

Task C: Login + Dashboard

1. `login.jsp`: HTML form (username, password).
2. `LoginServlet`: validate credentials. On success: store user in `sessionScope` and redirect to `dashboard.jsp`. On failure: set error message in `requestScope` and forward back to `login.jsp`.
3. `login.jsp`: show error message (if any) using `c:if` and `c:out`.
4. `dashboard.jsp`: display username from session. Show "Admin" panel only if `user.role == "ADMIN"` using `c:if`. Use `c:redirect` to login page if session is empty.

Constraint
No scriptlets allowed in any JSP file. Use EL and JSTL exclusively.

FPT Education
FPT UNIVERSITY

Classwork 5: Discussion Questions

1. Why is `${user.name}` safer than `<%= user.getName() %>`? Give two reasons.
2. What is the difference between `${param.username}` and `${requestScope.username}`? When would each be `null`?
3. Explain the **scope search order** in EL. Why might this lead to bugs? Give a concrete example.
4. When should you use `c:out` instead of `${value}`? What security risk does `c:out` protect against?
5. Compare `c:if` and `c:choose`. When is each appropriate?

FPT Education
FPT UNIVERSITY

Summary: Slot 13, 14 — EL & JSTL

Expression Language (EL)

- Syntax: `${expr}`
- Access beans, collections, parameters
- Operators: arithmetic, relational, logical, `empty`
- 11 implicit objects
- Null-safe (no exceptions)
- Scope search: page → request → session → app

JSTL

- 5 libraries: core, fmt, sql, xml, fn
- Core: `c:out`, `c:set`, `c:if`, `c:choose`, `c:forEach`, `c:url`
- Fmt: dates, numbers, i18n
- Fn: string functions
- Declare with `<%@ taglib %>`
- Use EL expressions in tag attributes

Up Next
Slot 15: Custom Tags — creating your own reusable JSP tags.

FPT Education
FPT UNIVERSITY

Questions?

FPT Education
FPT UNIVERSITY